

Report conversation

When I use `scanf("%lld", p);`, it shows warning on Linux. When I use `scanf("%ld", p);`, it shows warnings on macos. What should I do? I think it is related to gcc vs clang. `p` is declared as `int64_t *`.

The warnings you are encountering when using `scanf` are related to format specifiers not matching the type of the pointer you're using. To fix this issue and ensure your code is portable across different platforms (Linux with GCC and macOS with Clang), you should use the correct format specifier for `int64_t` and make sure you pass a pointer to the appropriate type.

On most platforms, including Linux with GCC and macOS with Clang, `int64_t` is a 64-bit integer type, so you should use the `%lld` format specifier. However, you need to pass a pointer to `int64_t` when using `scanf`. Here's how you can do it:

c

 Copy code

```
#include <stdio.h>
#include <inttypes.h>

int main() {
    int64_t value;
    int64_t *p = &value;

    // Use the correct format specifier and pass a pointer to int64_t
    if (scanf("%" SCNd64, p) == 1) {
        // Input was successfully read into the variable 'value'
        printf("Value: %" PRIu64 "\n", value);
    } else {
        printf("Input error\n");
    }

    return 0;
}
```

In this code, we use `SCNd64` for the format specifier, which is a macro from the `<inttypes.h>` header that corresponds to the correct format specifier for `int64_t`. Similarly, we use `PRId64` to print the value, which is the correct format specifier for `int64_t`.

This code should work without warnings on both Linux with GCC and macOS with Clang, ensuring portability across different platforms.